

Übungsblatt 6

Data Engineering

5430UE Web and Data Engineering

Prof. Dr. Michael Granitzer

24 März 2026

Modul A

Big Data Grundlagen

Übung 6.A.1: Batch vs. Streaming

Ordnen Sie folgende Anforderungen der geeigneten Verarbeitungsart zu (Batch / Streaming / beides möglich):

Anforderung	Batch / Streaming
Tagesabschlussrechnung für alle Bestellungen	?
Echtzeit-Benachrichtigung bei verdächtiger Kreditkartentransaktion	?
Monatlicher Aktivitätsreport für alle Nutzer	?
Lagerbestand aktualisieren sobald ein Produkt verkauft wird	?
Empfehlungsmodell wöchentlich neu trainieren	?

Begründen Sie die Zuordnung bei Streaming-Fällen: Warum reicht Batch hier nicht?

Lösung



Anforderung	Empfehlung	Begründung
Tagesabschluss	Batch	Ergebnis erst am Tagesende nötig; hoher Durchsatz wichtiger als geringe Latenz
Betrugserkennung	Streaming	Batch würde Betrug erst Stunden später erkennen — Schaden bereits entstanden
Monatsreport	Batch	Einmal monatlich, große Datenmenge — Batch-Effizienz ideal
Lagerbestand	Streaming	Überverkauf verhindern erfordert Echtzeitaktualisierung
Empfehlungsmodell	Batch	Training auf historischen Daten; Modell muss nicht nach jeder Aktion neu trainiert werden

Streaming-Argumente: Latenz-Anforderung < Batch-Intervall, Entscheidungen müssen getroffen werden bevor alle Daten vorhanden sind, Ereignisse haben Verfallsdatum.

Übung 6.A.2: Anwendungsbereiche von Big Data

Nennen Sie 4 verschiedene Anwendungsbereiche für Big Data. Beschreiben Sie für jeden Bereich:

a) Welche Daten werden gesammelt und wie? b) Welches der 5 Vs (Volume, Velocity, Variety, Veracity, Value) ist in diesem Bereich besonders herausfordernd? c) Welche Art von Analyse erzeugt Value aus den Daten?

Lösung

Bereich	Daten & Erhebung	Besonders herausfordernd	Value durch Analyse
E-Commerce	Clickstreams, Kaufhistorie, Suchbegriffe — automatisch protokolliert pro Session	Velocity (tausende Events/Sekunde) + Volume	Empfehlungssysteme, personalisierte Werbung, A/B-Testing
Smart Farming	IoT-Sensordaten (Bodenfeuchte, Wetter, Drohnenbilder) — kontinuierlich gesendet	Veracity (fehlerhafte oder ausgefallene Sensoren)	Ertragsvorhersage, optimierter Dünger-/Wassereinsatz



Bereich	Daten & Erhebung	Besonders herausfordernd	Value durch Analyse
Finanzwesen/Banking	Transaktionen, Marktdaten, Kreditscores — Echtzeit-Feeds	Velocity (Echtzeit-Betrugsschutz < 100 ms)	Fraud Detection, algorithmischer Handel, Risikomodellierung
Industrie / Predictive Maintenance	Maschinensensoren (Temperatur, Vibration, Laufzeit) — Edge-Devices	Variety (verschiedene Maschinentypen, heterogene Formate)	Ausfallvorhersage, geplante Wartung statt Notfallreparatur

Übung 6.A.3: CAP-Theorem

Das CAP-Theorem besagt, dass ein verteiltes System maximal zwei der drei Eigenschaften **Consistency**, **Availability** und **Partition Tolerance** gleichzeitig garantieren kann.

Ordnen Sie die folgenden Systeme einer CAP-Kombination zu und begründen Sie Ihre Entscheidung:

a) Traditionelle relationale Datenbank (z.B. PostgreSQL in einem Single-Node Setup) b) Apache Cassandra c) Apache Kafka

Lösung

System	CAP-Kombination	Begründung
PostgreSQL Single-Node	CA	Kein Netzwerkpartitionierungsszenario bei Single-Node. Strong Consistency (ACID-Transaktionen) und hohe Availability. Partition Tolerance ist irrelevant, da kein Cluster.
Apache Cassandra	AP	Cassandra ist über mehrere Knoten repliziert und bleibt bei Netzwerkpartitionen verfügbar. Konsistenz ist „tunable“ (eventual consistency per Default) — kein starkes ACID.



System	CAP-Kombination	Begründung
Apache Kafka	CP	Nachrichten innerhalb einer Partition sind streng geordnet und konsistent (Commit erst nach Replikation). Bei Leader-Election-Prozessen kann eine Partition kurzzeitig nicht beschreibbar sein — Availability wird geopfert.

Hinweis: In der Praxis ist Partition Tolerance bei verteilten Systemen nicht verzichtbar. Deshalb wählen verteilte Systeme in der Regel entweder AP oder CP.

Übung 6.A.4: Die 4 V's und Datenqualität

Ein E-Commerce-Unternehmen sammelt folgende Daten:

- Clickstream-Logs: 50.000 Events/Sekunde
- Produktrezensionen: mehrsprachig, strukturiert und als Freitext
- Bestelldaten: strukturiert, transaktional
- Sensor-IoT-Daten aus Lagerhäusern: teils fehlerhafte Messwerte

Ordnen Sie je einem der 4 V's (Volume, Velocity, Variety, Veracity) die größte Herausforderung zu und beschreiben Sie eine konkrete Maßnahme.

Lösung

V	Datenquelle	Herausforderung	Maßnahme
Volume	Clickstream 50k/s	Speicher- und Verarbeitungskapazität	Kafka als Message Buffer + Spark Streaming für Aggregation
Velocity	Clickstream	Echtzeit-Ingestion ohne Datenverlust	Partitioniertes Kafka-Topic, Consumer Groups mit Backpressure



V	Datenquelle	Herausforderung	Maßnahme
Variety	Produktrezensionen	Mehrsprachiger Freitext + strukturierte Felder	NLP-Pipeline für Sentiment, separate Schemata pro Typ
Veracity	IoT-Sensordaten	Fehlerhafte/fehlende Messwerte	Anomalie-Erkennung, Imputation fehlender Werte, Konfidenzscores

Modul B

Data Warehouse & Datenarchitektur

Übung 6.B.1: Datenbank vs. Data Warehouse

Erklären Sie die Unterschiede zwischen einer operativen Datenbank (OLTP) und einem Data Warehouse (OLAP) anhand folgender Dimensionen:

Dimension	OLTP	OLAP
Primäre Aufgabe	?	?
Typische Abfragen	?	?
Datenmenge pro Abfrage	?	?
Aktualisierungsfrequenz	?	?
Beispiel-Technologie	?	?

Warum kann man analytische Workloads nicht einfach auf der operativen Datenbank laufen lassen?

Lösung



Dimension	OLTP	OLAP
Primäre Aufgabe	Transaktionsverarbeitung (ACID)	Analytische Auswertungen (Aggregationen)
Typische Abfragen	INSERT, UPDATE, einfaches SELECT per PK	Komplexe JOINS, GROUP BY, Aggregatfunktionen über Millionen Zeilen
Datenmenge/Abfrage	Wenige Zeilen (eine Bestellung)	Millionen bis Milliarden Zeilen (Jahresumsatz)
Aktualisierungsfrequenz	Sekündlich (jede Transaktion)	Täglich/stündlich (ETL-Prozess)
Beispiel	PostgreSQL, MySQL	Snowflake, BigQuery, Redshift

Warum kein analytischer Workload auf OLTP? Schwere SELECT-Abfragen (Full Table Scans über Millionen Zeilen) sperren Tabellen und blockieren laufende Transaktionen. Das gefährdet die Verfügbarkeit des operativen Systems.

Übung 6.B.2: Star Schema

Ein Online-Shop betreibt ein Data Warehouse. Die Faktentabelle enthält jede Bestellung mit folgenden Maßzahlen: Bestellbetrag (€), Anzahl Artikel, Rabatt (%).

- a) Skizzieren Sie das Star-Schema für dieses Data Warehouse: Welche Dimensionstabellen werden benötigt und welche Attribute enthält jede Tabelle?
- b) Schreiben Sie eine SQL-Abfrage für: „Gesamtumsatz pro Produktkategorie pro Quartal im Jahr 2024“

Lösung

a) Star Schema

Faktentabelle:

```
FactBestellung(BestellID, KundeID, ProduktID, ZeitID, KanalID,  
              Betrag, AnzahlArtikel, Rabatt)
```

Dimensionstabellen:

Tabelle	Attribute
DimKunde	KundeID, Name, Ort, Land, Segment
DimProdukt	ProduktID, Produktname, Kategorie, Marke



Tabelle	Attribute
DimZeit	ZeitID, Datum, Monat, Quartal, Jahr, Wochentag
DimKanal	KanalID, Typ (Online / Mobile / Store)

Das Star-Schema erlaubt effiziente analytische Abfragen durch direkte JOINS von der Faktentabelle zu den Dimensionstabellen ohne weitere Normalisierungsebenen.

b) SQL-Abfrage

```
SELECT
  p.Kategorie,
  t.Quartal,
  SUM(f.Betrag) AS Gesamtumsatz
FROM FactBestellung f
JOIN DimProdukt p ON f.ProduktID = p.ProduktID
JOIN DimZeit t ON f.ZeitID = t.ZeitID
WHERE t.Jahr = 2024
GROUP BY p.Kategorie, t.Quartal
ORDER BY t.Quartal, p.Kategorie;
```

Modul C

Data Science & Data Mining

Übung 6.C.1: Deduktives vs. Induktives Reasoning

- a) Erklären Sie den Unterschied zwischen deduktivem und induktivem Reasoning. Geben Sie je zwei Beispiele.
- b) Welche typischen Fehlerquellen gibt es bei jedem Ansatz?
- c) Handelt es sich beim maschinellen Lernen um deduktives oder induktives Reasoning? Begründen Sie.

Lösung

a) Unterschied und Beispiele

Deduktives Reasoning — vom Allgemeinen zum Konkreten: Aus bekannten, allgemeinen Regeln (Prämissen) werden zwingend gültige Schlussfolgerungen abgeleitet.

- Beispiel 1: „Alle Menschen sind sterblich. Sokrates ist ein Mensch. → Sokrates ist sterblich.“
- Beispiel 2: „Wenn Körpertemperatur $> 38\text{ °C}$ UND Husten vorhanden → Grippe-Verdacht.“ (Regelbasierte Expertensysteme)



Induktives Reasoning — vom Konkreten zum Allgemeinen: Aus vielen beobachteten Einzelfällen werden verallgemeinerte Muster oder Regeln abgeleitet.

- Beispiel 1: Aus 1000 beobachteten Äpfeln, die vom Baum fallen, wird auf das Gravitationsgesetz geschlossen.
- Beispiel 2: Aus Kaufhistorien von Millionen Kunden werden Kaufmuster erkannt.

b) Fehlerquellen

Ansatz	Typische Fehlerquelle
Deduktiv	Falsche oder unvollständige Prämissen führen zu falschen Schlüssen — der Schluss ist logisch korrekt, aber inhaltlich falsch
Induktiv	Overfitting, Bias in den Beobachtungen (nicht-repräsentative Stichproben), Korrelation wird fälschlich als Kausalität interpretiert

c) Maschinelles Lernen ist induktives Reasoning: Das Modell lernt Regeln und Muster aus konkreten Trainingsdaten und leitet daraus verallgemeinerte Vorhersageregeln ab.

Übung 6.C.2: Data Mining Aufgaben klassifizieren

Ordnen Sie die folgenden Anwendungen der passenden Data-Mining-Aufgabe zu. Wählen Sie aus: **Klassifikation, Regression, Clustering, Assoziationsregeln, Collaborative Filtering**.

Geben Sie außerdem an, ob es sich um eine **prädiktive**, **deskriptive** oder **explorative** Aufgabe handelt.

a) Ein Bild wird analysiert und das System entscheidet: Hund oder Katze? b) Aus einem Foto eines Essens wird die Kalorienzahl geschätzt. c) Nutzer einer Streaming-Plattform werden anhand ihres Sehverhaltens in Gruppen eingeteilt, ohne vorgegebene Kategorien. d) Einem Nutzer werden Produkte vorgeschlagen, weil ähnliche Nutzer diese Produkte gekauft haben. e) Es wird ermittelt, welche Produkte im Supermarkt häufig zusammen in einem Einkaufskorb landen.

Lösung

#	Anwendung	Data-Mining-Aufgabe	Kategorie
a)	Hund/Katze erkennen	Klassifikation	Predictive
b)	Kalorienanzahl schätzen	Regression	Predictive



#	Anwendung	Data-Mining-Aufgabe	Kategorie
c)	Nutzer in Gruppen einteilen (ohne Labels)	Clustering	Explorative
d)	Produkte ähnlichen Nutzern empfehlen	Collaborative Filtering	Predictive
e)	Häufig gemeinsam gekaufte Produkte finden	Assoziationsregeln	Descriptive

Kategorien:

- **Predictive:** Vorhersage eines Zielwerts (Klasse oder kontinuierlicher Wert) für neue Datenpunkte
- **Descriptive:** Zusammenfassung oder Struktur bereits vorhandener Daten beschreiben
- **Explorative:** Unbekannte Strukturen oder Muster in Daten entdecken, ohne vorgegebene Zielvariable

Übung 6.C.3: CRISP-DM Phasen

Ein Webshop möchte vorhersagen, welche Kunden in den nächsten 30 Tagen abwandern (Churn Prediction).

Beschreiben Sie für jede CRISP-DM-Phase eine konkrete Aktivität für dieses Projekt:

1. Business Understanding
2. Data Understanding
3. Data Preparation
4. Modeling
5. Evaluation
6. Deployment

Warum ist CRISP-DM ein **iterativer** Prozess? Geben Sie ein Beispiel, wann man in eine frühere Phase zurückkehren muss.

Lösung



1. **Business Understanding:** Ziel definieren — Recall minimieren (jeden Abwandernden erkennen), Kosten eines falsch-negativen Ergebnisses bestimmen, Akzeptanzkriterium festlegen (z.B. Recall $\geq 80\%$).
2. **Data Understanding:** Welche Daten stehen bereit? Bestellhistorie, Login-Frequenz, Supporttickets, letzte Aktivität. Klassenungleichgewicht prüfen.
3. **Data Preparation:** Ableitungsfeatures berechnen (Tage seit letztem Kauf, Bestellfrequenz), fehlende Werte imputieren, Kategorien encodieren, Trainings-/Test-Split mit zeitlicher Trennung (kein Data Leakage).
4. **Modeling:** Baseline mit Logistic Regression, dann Gradient Boosting (XGBoost). Hyperparameter-Tuning mit Cross-Validation.
5. **Evaluation:** Recall, Precision, F1 auf Testdaten. Confusion Matrix. ROC-AUC. Vergleich mit Heuristik-Baseline.
6. **Deployment:** Modell als REST-API, täglicher Batch-Job schreibt Churn-Scores in CRM, Marketing-System triggert Retention-Kampagne.

Iterativer Charakter: In Phase 5 (Evaluation) stellt man fest, dass das Modell bei einem Kundensegment schlecht abschneidet. Das führt zurück zu Phase 3 (neue Features) oder sogar Phase 2 (neue Datenquellen). Der Prozess endet erst, wenn das Modell die definierten Geschäftsziele erfüllt.

Modul D

MapReduce & Hadoop

Übung 6.D.1: MapReduce Reducer analysieren

```
import sys
current_word = None
current_count = 0
for line in sys.stdin:
    line = line.strip()
    word, count = line.split('\t', 1)
    try:
        count = int(count)
    except ValueError:
        continue
    if current_word == word:
        current_count += count
    else:
        if current_word:
            print('%s\t%s' % (current_word, current_count))
            current_count = count
            current_word = word
        if current_word == word:
            print('%s\t%s' % (current_word, current_count))
```

a) Was macht dieser Code? Erklären Sie die Logik. b) Warum muss die Eingabe **sortiert nach Wort** sein, damit der Code korrekt funktioniert? c) Was ist die Ausgabe für die Eingabe (sortiert): Hallo\t1, Hallo\t1, Welt\t2?

Lösung

a) Logik des Reducers

Der Code liest zeilenweise Tab-separierte (Wort, Zahl)-Paare von stdin. Er hält das aktuelle Wort und einen laufenden Zähler. Solange das gleiche Wort in der nächsten Zeile erscheint, wird der Zähler addiert. Sobald ein neues Wort erscheint, wird das vorherige Wort mit seinem aggregierten Zähler ausgegeben. Funktion: **Aggregation von Häufigkeitszählungen identischer Wörter**.

b) Warum sortierte Eingabe notwendig?

Der Reducer hält immer nur den Zähler für das *aktuelle* Wort im Speicher. Er erkennt den Wechsel zu einem neuen Wort nur anhand der Reihenfolge. Ohne Sortierung würde er ein Wort, das an verschiedenen Stellen vorkommt, als zwei verschiedene Wörter behandeln und separat ausgeben.

In Hadoop/MapReduce übernimmt die **Shuffle-and-Sort Phase** diese Sortierung automatisch.

c) Erwartete Ausgabe

```
Hallo 2  
Welt 2
```

Übung 6.D.2: MapReduce Mapper schreiben

Schreiben Sie einen Python Mapper (map.py), der Sätze von stdin einliest und für jedes Wort eine Zeile der Form `WORT\t1` ausgibt. Berücksichtigen Sie:

- Großschreibung ignorieren (alles in Kleinbuchstaben)
- Satzzeichen am Wortende entfernen (., !?; :)
- Leere Wörter überspringen

Testen Sie den kompletten Workflow mit:

```
echo "Ein Haus an einem Haus könnte ein kleines Haus oder eine Garage sein." \  
| python map.py | sort -k1,1 | python reduce.py
```

Was ist die erwartete Ausgabe?

Lösung

Mapper (map.py)

```
#!/usr/bin/env python  
import sys  
for line in sys.stdin:  
    line = line.strip().lower()  
    words = line.split()  
    for word in words:  
        word = word.strip('.,!?:;:')  
        if word:  
            print('%s\t%s' % (word, 1))
```



Erwartete Ausgabe (alphabetisch sortiert):

```
an 1  
ein 2  
einem 1  
eine 1  
garage 1  
haus 3  
kleines 1  
könnte 1  
oder 1  
sein 1
```

sort übernimmt hier die Rolle der Shuffle-and-Sort Phase, die in Hadoop automatisch zwischen Map- und Reduce-Phase stattfindet.

Übung 6.D.3: Hadoop HDFS Konzepte

- a) In welche Komponenten ist HDFS aufgeteilt und was ist die Aufgabe jeder Komponente?
- b) Eine Datei wird in HDFS mit einem Replikationsfaktor von 3 gespeichert und hat eine Größe von 200 MB (Chunk-Größe: 64 MB). Wie viele Chunks entstehen und wie viele Chunk-Kopien existieren insgesamt? Wie viel Speicher wird insgesamt belegt?
- c) Was bedeutet **Data Locality** und warum ist es für die MapReduce-Performance entscheidend?

Lösung

a) HDFS-Komponenten

Komponente	Aufgabe
NameNode (Master)	Speichert Metadaten: Welche Datei besteht aus welchen Chunks? Wo liegen die Replikate? Verwaltet den Namespace.
DataNodes (Chunk Server)	Speichern die eigentlichen Daten-Chunks auf lokalem Dateisystem. Melden regelmäßig ihren Zustand (Heartbeat) an den NameNode.
Client Library	Kommuniziert mit dem NameNode für Chunk-Locations, dann direkt mit den DataNodes zum Lesen/Schreiben.

b) Chunks und Replikate



- Anzahl Chunks: $\lceil 200 \div 64 \rceil = 4$ Chunks ($3 \times 64 \text{ MB} + 1 \times 8 \text{ MB}$)
- Mit Replikationsfaktor 3: $4 \times 3 = 12$ Chunk-Kopien im Cluster
- Gesamter Speicherbedarf: $200, \text{MB} \times 3 = 600, \text{MB}$

c) Data Locality

Data Locality bedeutet, dass MapReduce-Tasks bevorzugt auf demselben Knoten ausgeführt werden, auf dem der benötigte Daten-Chunk physisch gespeichert ist.

Bedeutung für Performance: Bei großen Datenmengen ist das Verschieben der Daten über das Netzwerk sehr teuer. Data Locality vermeidet diesen Netzwerktransfer („Move Computation, not Data“). Wenn Data Locality nicht möglich ist, versucht Hadoop zunächst einen Knoten im gleichen Rack zu verwenden (Rack Locality).

Modul E

Statistik und Machine Learning Grundlagen

Übung 6.E.1: Random Sampling — Einfach vs. Stratifiziert

Ein Forschungsteam möchte eine Stichprobe von 500 Studierenden aus einer Hochschule mit 5.000 Studierenden ziehen.

Fakultät	Studierende	Anteil
Informatik	1.000	20%
Wirtschaft	2.000	40%
Ingenieurwesen	1.500	30%
Geisteswissenschaften	500	10%

- 1. Wie viele Studierende kämen bei **einfacher Zufallsstichprobe** aus jeder Fakultät? Ist das ein Problem?
- 2. Wie viele Studierende würden bei **stratifizierter Stichprobe** aus jeder Fakultät gezogen? Welcher Vorteil?
- 3. Wann ist einfache Zufallsstichprobe ausreichend, wann ist Stratifizierung notwendig?

Lösung

1. **Einfache Zufallsstichprobe** — jeder Studierende mit Wahrscheinlichkeit $\frac{500}{5000} = 10\%$:

Fakultät	Erwartete Stichprobengröße
Informatik	100



Fakultät	Erwartete Stichprobengröße
Wirtschaft	200
Ingenieurwesen	150
Geisteswissenschaften	~50

Problem: Bei kleinen Fakultäten kann die Stichprobengröße zu gering für statistische Aussagen sein. Zufallsschwankungen machen Schätzungen instabil.

2. Stratifizierte Stichprobe — proportional aus jedem Stratum:

Fakultät	Stichprobengröße
Informatik	$500 \times 0.20 = 100$
Wirtschaft	$500 \times 0.40 = 200$
Ingenieurwesen	$500 \times 0.30 = 150$
Geisteswissenschaften	$500 \times 0.10 = 50$

Vorteil: Jede Teilgruppe ist proportional repräsentiert. Schätzungen innerhalb jeder Fakultät sind stabiler. Wichtig wenn Subgruppenvergleiche gewünscht sind.

 **3. Wann welches Verfahren?**

Übung 6.E.2: Machine Learning — Wahrscheinlichkeit und Klassifikation

Ein Modell zur Erkennung handgeschriebener Ziffern wird auf 10.000 Testbildern evaluiert.

Gegeben:

- Gesamtgenauigkeit (Accuracy): 97 %
- Das Modell klassifiziert die Ziffer „4“ in 95 % aller Fälle korrekt
- Die Ziffer „4“ macht 10 % aller 10.000 Testbilder aus
- 8 % der als „4“ klassifizierten Bilder sind tatsächlich keine „4“

1. Wie viele der 10.000 Testbilder zeigen eine „4“? Wie viele werden korrekt erkannt?
2. Was ist der Unterschied zwischen **Precision** und **Recall**? Berechnen Sie Precision für die Klasse „4“.
3. Warum ist **Accuracy** alleine kein ausreichendes Gütekriterium bei ungleicher Klassenverteilung?

Lösung

1. Absolute Zahlen:

- Bilder mit Ziffer „4“: $10.000 \times 0.10 = 1.000$
- Korrekt als „4“ erkannt (True Positives): $1.000 \times 0.95 = 950$

2. Precision vs. Recall:



- **Recall (Sensitivity):** Von allen echten „4“-Bildern: wie viele wurden als „4“ erkannt? — $\frac{TP}{TP+FN}$ — Recall „4“ = 95
- **Precision:** Von allen als „4“ klassifizierten Bildern: wie viele sind wirklich eine „4“? — $\frac{TP}{TP+FP}$ — 8 % falsch → Precision = $1 - 0.08 = 92$

3. Accuracy und Klassenungleichgewicht

Beispiel: In einem Betrugserkennungsdatensatz sind 99 % der Transaktionen legitim. Ein Modell, das **immer** „legitim“ vorhersagt, erreicht 99 % Accuracy — obwohl es keinen einzigen Betrugsfall erkennt.

Lösung: Precision, Recall und F1-Score separat für jede Klasse betrachten. Der **F1-Score** ($= 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$) ist bei unbalancierten Klassen ein besseres Gesamtmaß.